Lab 3: Architecture Selection & Justification

Book Club Reading Challenge & Discussion Portal

NAME: ANANYA SURESH

SRN: PES2UG24CS059

CLASS:5A

Architecture Selection: We chose Microservices Architecture for the Book Club Reading Challenge & Discussion Portal System.

Reason 1 : Independent Scaling for Bursty Voting Load

NFR-001 requires the monthly poll to update tallies in real time and reject duplicate votes even under peak load. Voting traffic is highly bursty, spiking only near the poll deadline while remaining idle the rest of the month, whereas reading-goal tracking and discussion browsing experience steady, low-intensity traffic throughout. By using a Microservices Architecture, the Poll Voting Service can be scaled out independently during peak voting windows without over-provisioning the rest of the portal, something a single monolithic deployment would handle inefficiently.

Reason 2: Independent Deployment and Fault Isolation

The Discussion Thread Service, which handles spoiler tagging, chapter threads, and comment rendering, changes frequently and is used constantly, while the Poll Voting Service is only active once a month. Splitting these into separate services ensures that a bug, deployment, or outage in one (for example, a faulty spoiler-reveal update) does not affect poll voting or reading-goal tracking. It also allows each service to be released on its own schedule without requiring a full-system redeploy.

Security Advantage

Isolating the Poll Voting Service behind its own service boundary allows duplicate-vote prevention (FR-005, NFR-001) to be enforced at a single, tightly scoped choke point its interface with the external Session Verification System. This design concentrates all vote-integrity logic in one auditable service instead of scattering it across the codebase, ensuring that a compromise in lower-security components like the Reading Goal Service or Discussion Thread Service cannot bypass session verification or forge a vote.

Performance Benefit

Because the Discussion Thread Service is deployed and scaled independently, it can be allocated dedicated resources such as comment-table indexing and caching to meet NFR-002 (rendering a 500-comment thread in under 3 seconds). This prevents it from competing for CPU or database connections with the Poll Voting Service during high-traffic monthly voting periods. In a monolithic system, a spike in voting traffic could slow down thread rendering for all members browsing discussions at the same time.